

What is ML Engineering?

A broad overview of the field, how it compares to software engineering, its relationship to AI, and a deepdive into how an ML-powered app works.



GERGELY OROSZ

APR 25, 2023 • PAID



11



Share



Heads up: I'm on my spring break, as per [my holiday schedule](#). This means that this week and next week I'm publishing issues only on Tuesdays, and there is no Thursday, The Scoop issue. I'll be back on reflecting on recent events with renewed energy after!

Q: “As a software engineer, I’m interested in machine learning (ML.) Could you give an overview of this field, and some basics worth knowing about?”

Machine learning is a hot topic and the popularity of this field is only growing, especially with the recent focus on large language models (LLMs) and the huge buzz about AI. *It was just last week that we covered [The productivity impact of AI coding tools](#).*

For an overview of what machine learning is, I turned to [Vicki Boykis](#), a longtime machine learning engineer, who's been in the machine learning/data space for over a decade. She is currently a Senior Machine Learning Engineer at Duo Security, and was previously at Automattic (Tumblr, WordPress) and has worked as an ML consultant. Vicki writes [a tech newsletter](#), a [blog](#), and has organized a non-traditional and very interesting tech conference called [Normcore Tech Conference](#), which took place last December.

In this issue, Vicki covers:

1. Her background

2. What is machine learning?

2. What is machine learning?
3. A brief history of machine learning
4. How do ML projects work?
5. Machine learning and "Artificial Intelligence" (AI.) How do they relate?
6. A deep dive into how an ML-powered app works
7. What does the ML landscape look like, today?

With that, it's over to Vicki:

1. My background

Today, I work on building out machine learning infrastructure at Duo Security. My previous job was at Automattic, the parent company of WordPress.com, where I worked on search and recommendations problems both on WordPress.com and on Tumblr. Tumblr is the social media platform Automattic acquired back in 2019.

Industrial machine learning is a fairly young and evolving field, and like many in the industry, I didn't start out doing ML.

I graduated with a B.S. in economics and a few years later completed an MBA. My first jobs out of college were heavy on data analysis; I started out as an economic consultant, implementing econometric models for international trade with statistical packages including [SAS](#) (data management and analytics software,) [R](#) (the programming language,) and [Eviews](#) (a statistical data package for Windows.) I did all my data analysis work locally, using Microsoft Access, with lots of Excel spreadsheet manipulation for client deliverables. While I learned the fundamentals of what later became known as data science, I was still very far from doing engineering.

My next job as a data analyst at a major American telecom was all of a sudden in an environment where I had access to a lot of data. We had so much data I couldn't work with it locally anymore; everything was in Oracle and I needed to learn to write clean, optimized SQL queries which didn't hang or compete for resources with those written by fellow

analysts. I learned the ins and outs of SQL, how to query and analyze data, and the pain of dealing with null and missing values.

As a data analyst, I still worked mostly in SQL, but I was now part of an engineering team, which meant learning the cadence of engineering releases, sprints, and being exposed to production best practices. Sometime into my tenure, the organization decided to go all-in on what was then the latest "hotness," a technology with the funny name, "[Hadoop](#)." It allowed us to process volumes of streaming log data that our relational database infrastructure couldn't handle.

When I wanted to learn more than just the outputs of the system presented to me as piles of text files in HDFS, I moved upstream from writing [Apache Pig](#) scripts, into the guts of Hadoop itself, into HDFS and Storm topologies. I started working less with SQL and more with shell scripting and Python. This was during the time of the dawn of the "Big Data" era, and working in it meant learning concepts like lambda architectures, and writing convoluted MapReduce logic on the Java Virtual Machine (JVM).

Before we continue, here's an explainer of some terms mentioned above:

- [Hadoop](#): a framework to process large, distributed data sets across a network of computers. It consists of a storage part – known as Hadoop Distributed File System (HDFS) – and a processing part using the [MapReduce](#) programming model. Simply put, Hadoop splits files into large blocks, distributes them across nodes in a cluster, then transfers code packaged as Java archives (JAR archives) to process large amounts of data in parallel.
- [Apache Pig](#): a platform for analyzing large data sets. It was built to make it easier to work with Hadoop's MapReduce framework, allowing developers to focus more on analyzing bulk data sets and spend less time writing Map-Reduce programs.
- [Apache Storm](#): a distributed stream processing computation framework. A Storm application is designed as a "topology," similar to a graph. The edges of the graph are called streams, and they direct data from one node to another, describing a real-time data transformation pipeline.

- [Lambda architecture](#): a way to process massive quantities of data ("Big Data,") by taking advantage of both batch and stream-processing methods. This approach balances latency, throughput and fault tolerance.

After being exposed to all these toolsets, I moved from pure data analysis into the realm of software engineering. Having previously worked reactively with data presented to me by upstream systems over which I had no control, I was finally empowered to write my own code to get my own data.

Being in control of a system end-to-end and writing [imperative code](#), versus declarative-only SQL, appealed to me. At my next job as a data scientist at a SaaS product in financial services, I used the engineering skills I'd learned to build a warehouse for data modeling, and also in parallel, went deeper on ML theory, working with [Monte Carlo](#) simulations to model investments and building [decision tree models](#) to predict customer churn.

I then went into consulting, which exposed me to a wide variety of production-grade ML systems at Fortune 500 companies. Just as important, I learned what it means to have a product that works end-to-end, starting with defining customer needs. One major project was working on recommendations for a streaming platform similar to Netflix, which tracked user activity and showed relevant, personalized recommendations based on previous activity. At that point, I became very interested in both end-to-end ML systems and systems where the main machine learning task was information retrieval, i.e. trying to filter a lot of information to serve only relevant items to end-users.

With several data jobs under my belt, I realized three critical patterns of ML system design, to which I return over and over again in the course of my work:

1. It's impossible to build a ML system if you don't have clean data.
2. It's also not possible to build an ML system if you don't have the infrastructure and organizational support to do actual ML.
3. A machine learning model that doesn't go into production is not a valuable model, but you might not reach production on the first try. Good ML engineering practices rely very heavily on having time and space in a product release cycle to experiment and

very heavily on having time and space in a product release cycle to experiment and test assumptions.

2. What is Machine Learning?

Over the course of my career, I've worked at a range of companies, from tiny startups to corporations with over 100,000 employees. At these places I tackled a great variety of projects, including:

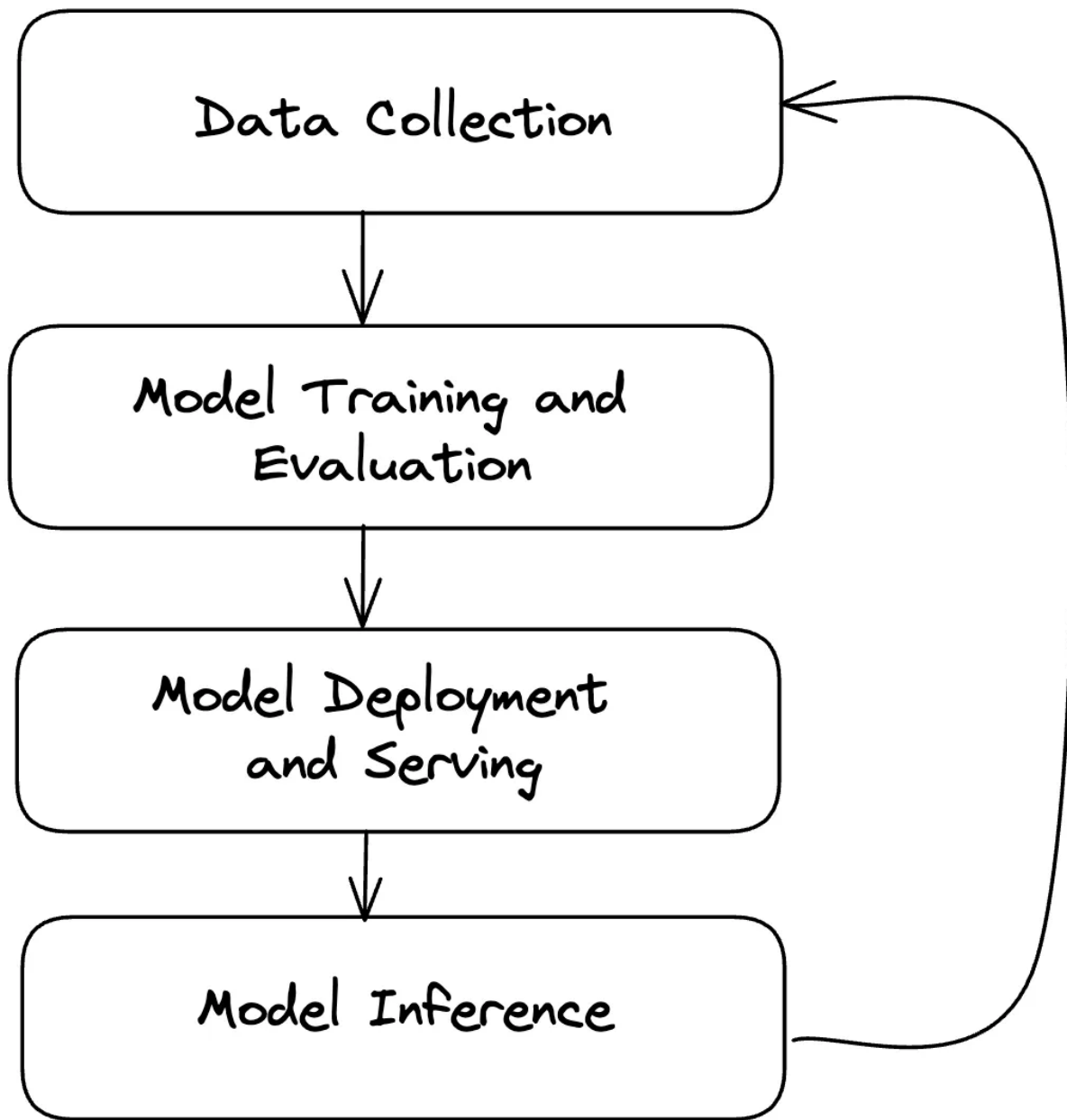
- Building recommendation systems for millions of users
- ML platforms in security
- Data transformations in healthcare
- Textual predictive modeling in patent law
- Kubernetes in quick-service restaurants

If you're reading these job descriptions and thinking: "that's just data analysis!" "that's just data science!" or "that's just engineering," then you're right; they do all fall under the umbrella of MLE (machine learning engineering) work.

Machine learning engineering is a rapidly growing field encompassing backend software engineering, machine learning algorithms, and analytics and statistics. It combines traditional software engineering practices with domain-specific skills: those required to develop, deploy, and maintain ML models on ML platforms. That's the general shape of the field, but the job looks very different depending on which organization you're in, the business area of expertise, and the maturity of the tech stack.

What do we mean by ML platforms? They're a collection of software components that create a systematic, automated way to take raw data, transform it, learn a model from it, and show results which support decision-making for internal or external customers.

A Machine Learning Platform



pragmaticengineer.com

Typical components within a machine learning platform

For example, [Netflix's recommendations](#) are part of ML platform. When you call an [Uber](#) or [Lyft](#), or book an [Airbnb](#), the chances are you're being served software components developed by those companies' internal ML organizations. These are very visible, consumer end-uses of machine learning, but ML is also used in the less visible parts of ML

organizations, such as [fraud detection at Stripe](#) and [at banks](#), and in content moderation on [social media platforms](#).

MLE requires a range of cross-disciplinary skills that encompass the full spectrum, from prototyping models to delivering reports. However, holistically, MLE is a job which sits firmly in the software engineering family.

3. A brief history of machine learning

Just as my career has moved from analytics to deploying complex distributed systems, the ML industry has grown from solving problems at the scale of generating reports, to being put into production at scale in the largest companies in tech today.

The early history of computing – and machine learning as a subset – was built on mainframes, which are enormous data servers that processed and kept track of data they generated in isolation from each other. In the 1970s, university researchers started configuring these standalone machines to talk to one another. [ARPANET](#) could be seen as the predecessor of the Internet. Here's what researchers at the University of Utah [wrote](#) in 1970, after connecting the university's computers to ARPANET:

"We have found that, in the process of connecting machines and operating systems together, a great deal of rapport has been established between personnel at the various network node sites. The resulting mixture of ideas, discussions, disagreements and resolutions, has been highly refreshing and beneficial to all involved, and we regard human interaction as a valuable by-product of the main effect."

When compute moved from machines to the network, the creators of these systems started keeping track of data movement and logging it, to get a consolidated view of data flows across their systems. Companies were already retaining analytical data needed to run critical business operations in relational databases, but access to that data was structured and processed in batch increments on a daily or weekly basis. This new logfile data moved quickly, and with a level of variety absent from traditional databases.

Capturing log data at scale began the rise of the Big Data era, which resulted in a great deal of variety, velocity, and volume of data movements. [The Apache logfile](#) was one of the greatest enablers of Big Data. The rise in data volumes coincided with data storage becoming much cheaper, enabling companies to store everything they collected on racks of commodity hardware.

When the data science boom hit in the early 2010s, the initial focus was on collecting as much data as possible, understanding a company's datasets and creating models from them.

Once companies realized they sat on potential goldmines of unstructured data – none of which were actionable – they started hiring data scientists to make sense of it all. The practice of [modern data science arose](#) from statisticians who observed the amount of data being generated and processed required methods beyond the scope of academic statistics and processing methods on single machines.

And so data scientists specialized in building models and making sense of piles of data. However, when it came to creating model artifacts that could continuously run, and at scale, data scientists relied on the help of production engineers to rewrite models in production-facing languages. At this stage, there was a very split hierarchy:

1. **Data engineers** who built systems to process and manage data at scale. *Read more about [what data engineering is in this article](#).*
2. **Data scientists** who created the models.

This split became known as the 'A/B divide in data science,' [A and B: analyst and builder](#): the people who analyzed the data or built models, and those who built systems around those models. As companies began to understand and expand the use of ML, and the split between A and B data scientists became more refined, and the data engineer and data scientist roles subdivided into several new, even more specialized roles:

- **Data engineering:** responsible for engineering the pipelines that bring data, schema enforcement, and streaming data

- **Data analysts/data scientist:** Analyze data for organizational needs, build dashboards, and present analyses
- **Analytics engineer:** Somewhere between data engineering and analysts, focuses on enforcing data movement between analytics tools and relational databases
- **Researcher:** Build conceptual models, test them, responsible for research and scientific discovery
- **Machine learning engineer:** Focus on building the models and productionalizing them, regardless of model type: anything from simple linear regression to ChatGPT

At the same time, model artifacts and deliverables moved from pure one-off analyses that lived on individuals' laptops, to production software such as the services powering web applications like Amazon and Netflix's recommendations, risk scoring for fraud, and medical diagnostic tools. This software required models be portable, low-latency, and be managed in a central place, which meant building systems and platforms on which to manage them. The term "MLOps" arose to define the boundaries of model management and operationalization.

It became clear that in order to successfully traverse this landscape, a data scientist needed to be a "[person who is better at statistics than any software engineer](#) and better at software engineering than any statistician."

To solve the complex problems of coordination, analysis, and engineering that machine learning in production involves, many companies and teams worked on in-house platform approaches which were open-sourced as top-level Apache projects, or turned into tools built by vendors.

Examples in the ML ecosystem are:

- [Apache Spark](#): a distributed batch computational platform processing large quantities of data that's still widely used today
- [Apache Airflow](#): a scheduler for data engineering pipelines and data orchestration
- [Apache Lucene](#): a search engine. [Elasticsearch](#) is one of the most popular distributed

and horizontally scalable search frameworks built on top of Apache Lucene.

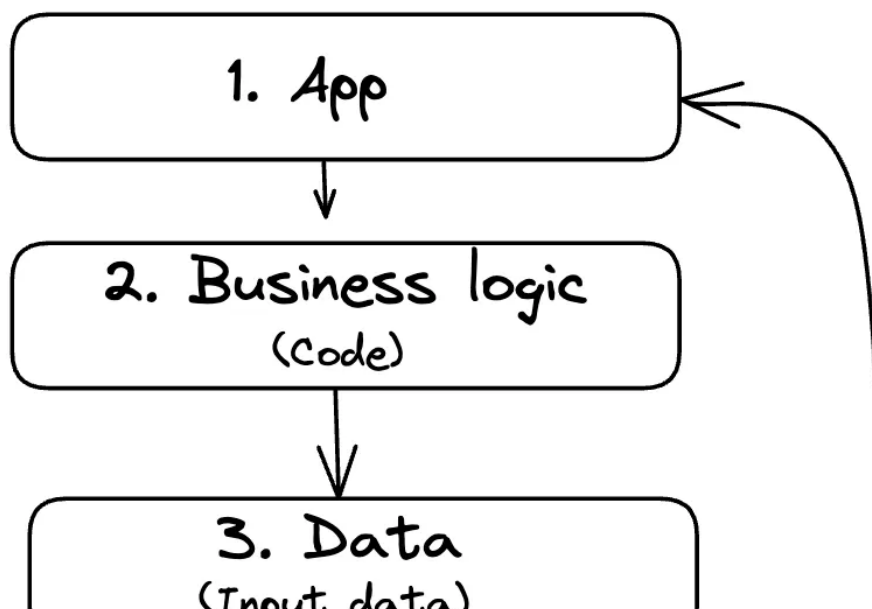
Today, the ML landscape is enormously wide and deep [in terms of tooling and approaches](#), and it's the MLE's job to make sense of them all in order to deploy production-grade models.

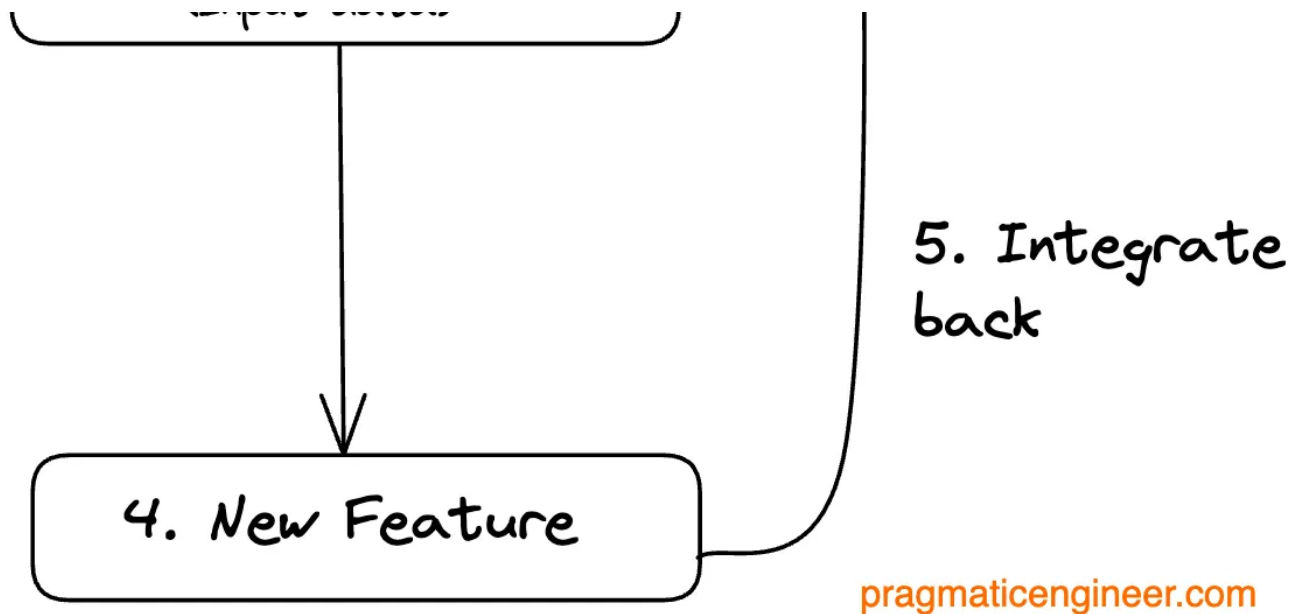
4. How do ML projects work?

Machine learning projects differ from general software development in their workflows. In a typical SaaS product, the user-facing app is made up of product features written in code. Product features are a broad concept, but are generally things that change the functionality of an app, like the "home timeline" or "feed of recommendations," or the "ability to post images".

When we think of a typical software development lifecycle, it's roughly this:

Traditional software engineering feature development flow





Typical software engineering feature development flow

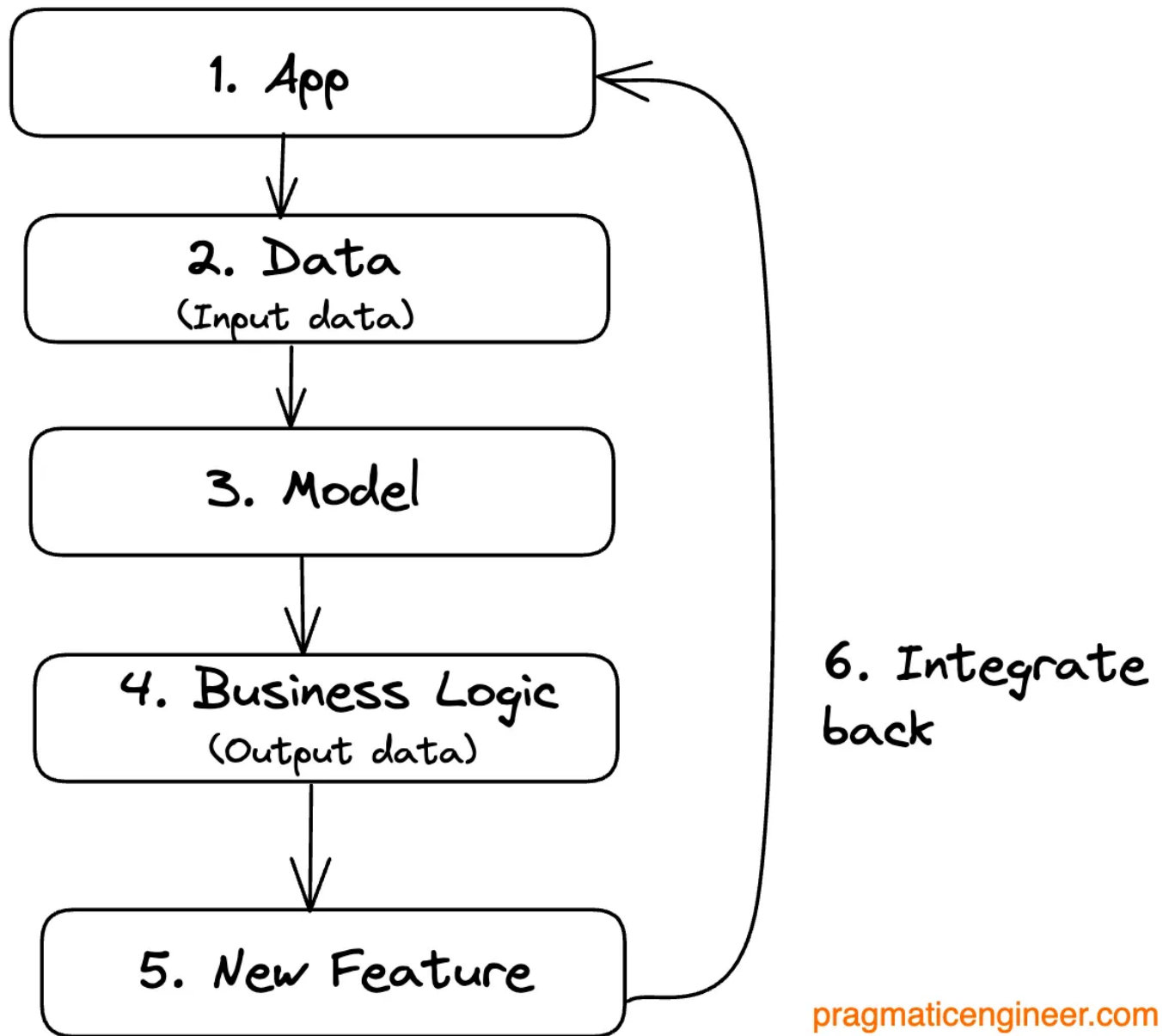
To add a new feature:

1. Take an existing app
2. Add business logic, usually as a form of code
3. This logic uses data already present in the app
4. The new feature uses this data and business logic
5. Add the new feature back into the application.

The business logic we add may not always be entirely straightforward, but the process itself is.

However, the process is different for applications powered by machine learning, for which we invert the workflow:

ML engineering feature development flow



How feature development typically works in MLE

The steps:

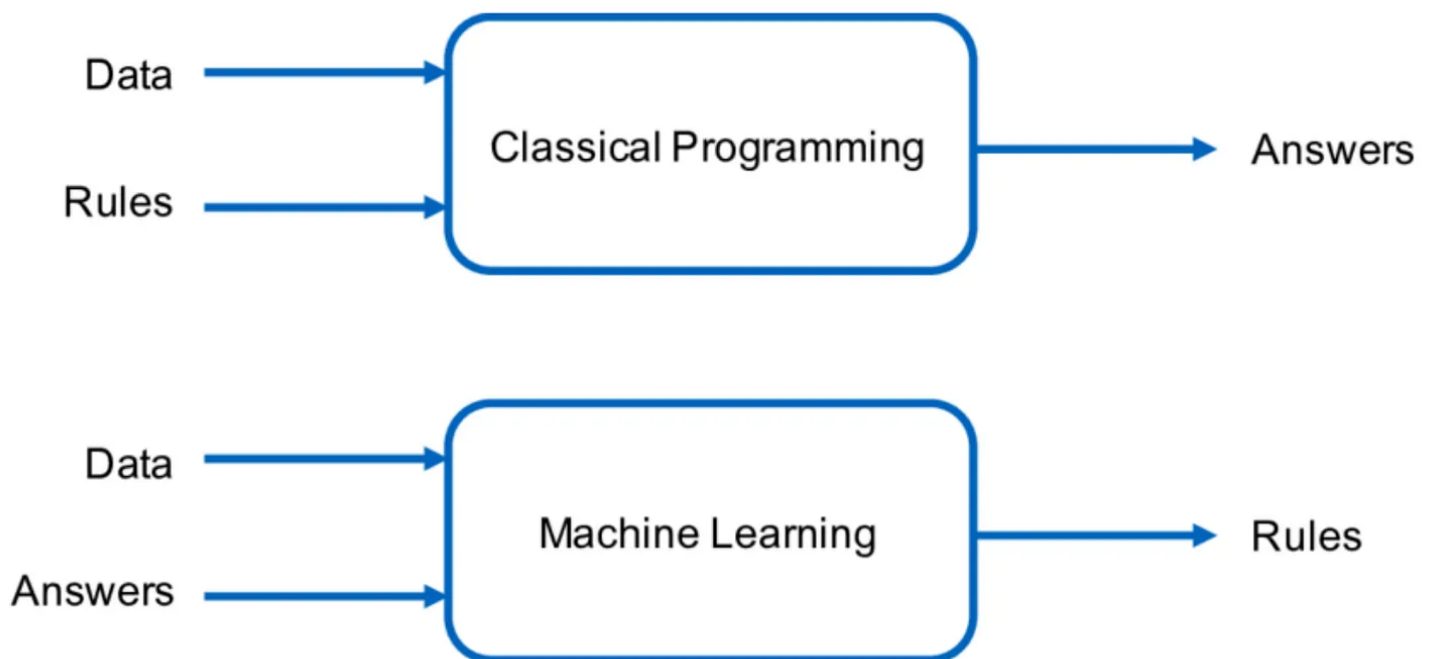
1. We still start with the app
2. However, we don't start with business logic, such as the code that generates a button's toggle logic. Instead, we use input data to:
3. Build a model

4. This model generates output data, which can be many things: content generated, or perhaps a decision, such as when to send an Uber
5. Integrate the output data into the new product feature
6. Integrate the new feature back into the application

ML development inverts the traditional software engineering development flow.

Instead of using business logic on input data, we generate the business logic *from* input data. This business logic is generated by applying the input data to a model, which then generates the output data which is the business logic.

With traditional applications, we feed them data and rules and get answers as output. In ML applications, we feed them data, specify what the answers should look like and get the rules in the form of the model, which we then use to make new predictions for developing new product features. Here's an illustration of this approach from the book, [Deep Learning with Python](#), by François Chollet:



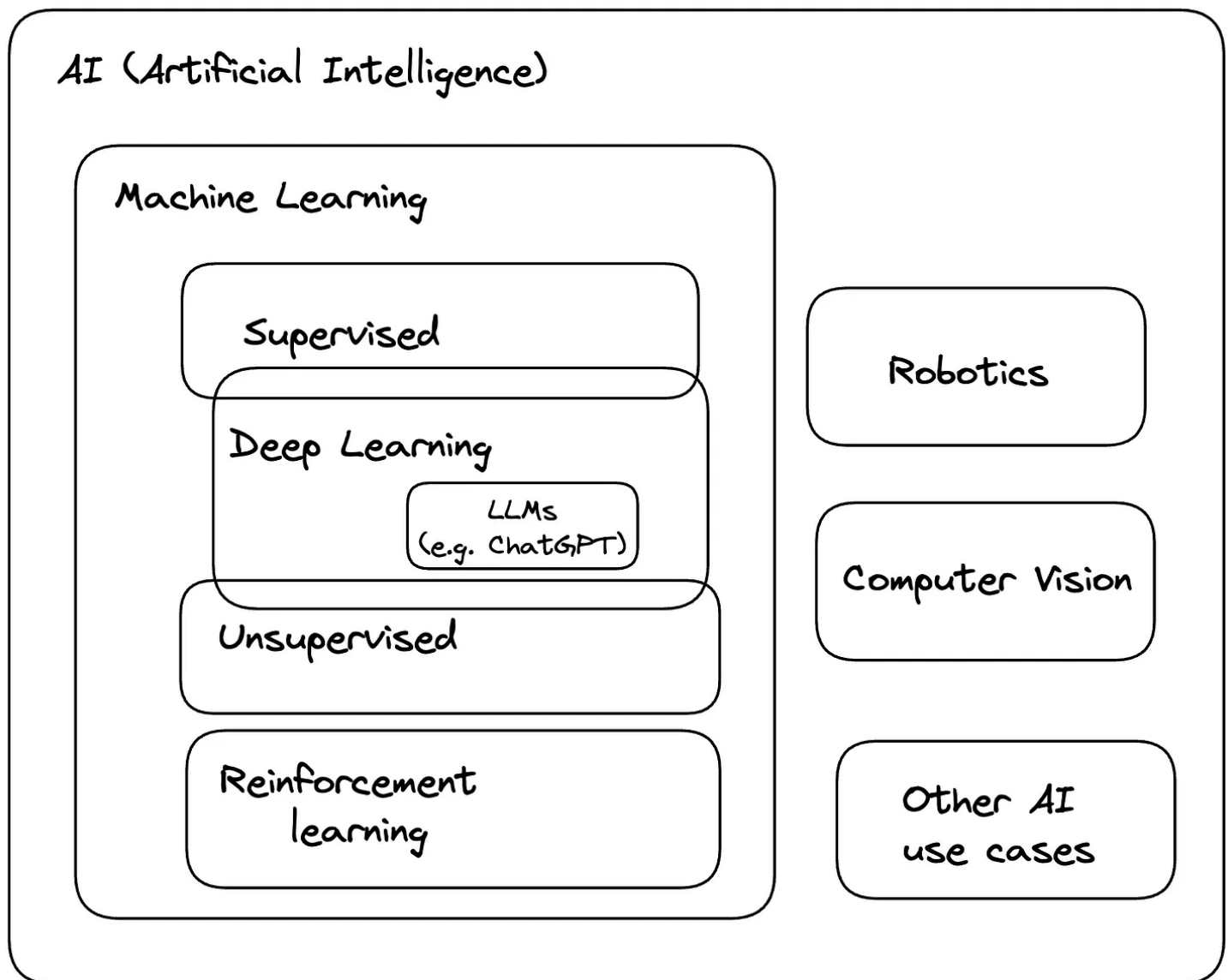
The difference between classical programming and machine learning. Image source: [Deep Learning with Python](#)

5. Machine learning and “Artificial Intelligence”

So far, we’ve talked about ML fundamentals, concepts, and platforms. These days, the amount of buzz and interest around AI is intense, so it’s worth contextualizing how this work fits into AI.

“AI” is a blanket term for any technology using machines to perform human tasks.

Machine learning is a subset of AI that uses machines to infer patterns from data.



pragmaticengineer.com

Mental model of the AI and Machine Learning landscape

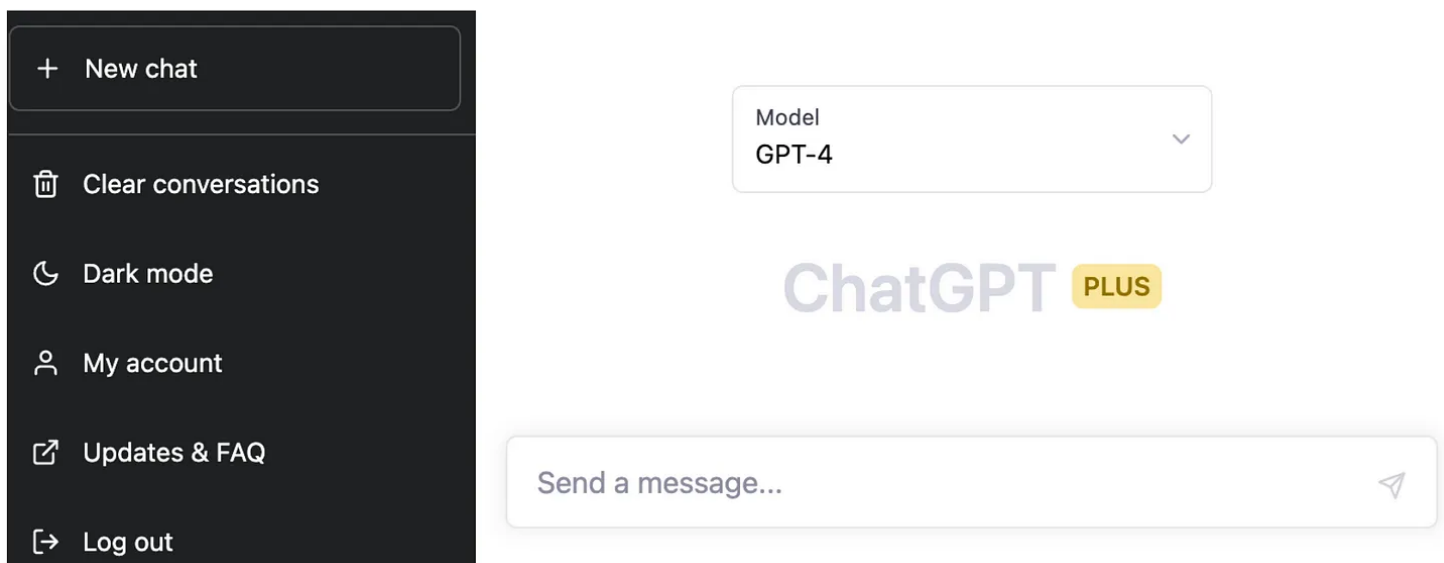
There are many different ways to do ML. One subset is [deep learning](#), which uses [neural networks](#), of which [large language models \(LLMs\)](#) such as [ChatGPT](#) are a specific type, which work with language tasks to perform ML, and more generally, AI.

With the explosive advances in [generative AI](#), [predictive text](#), and [Transformer-based models](#), coupled with the growth of GPU-based computing, deep learning has started to make a dent in Big Tech and at leading companies in the tech industry. Despite this, no matter how buzzy AI is right now, it still only makes up a tiny – though rapidly growing – fraction of ML use-cases in industry.

6. A deep dive into how ChatGPT works

We've discussed high-level concepts so far, which may feel abstract. To get more hands-on, let's take a look at a practical example of how an ML-powered application works. We'll use the very popular tool, ChatGPT, as an example. Let's look at what a ChatGPT app looks like, from the point of view of an ML engineer.

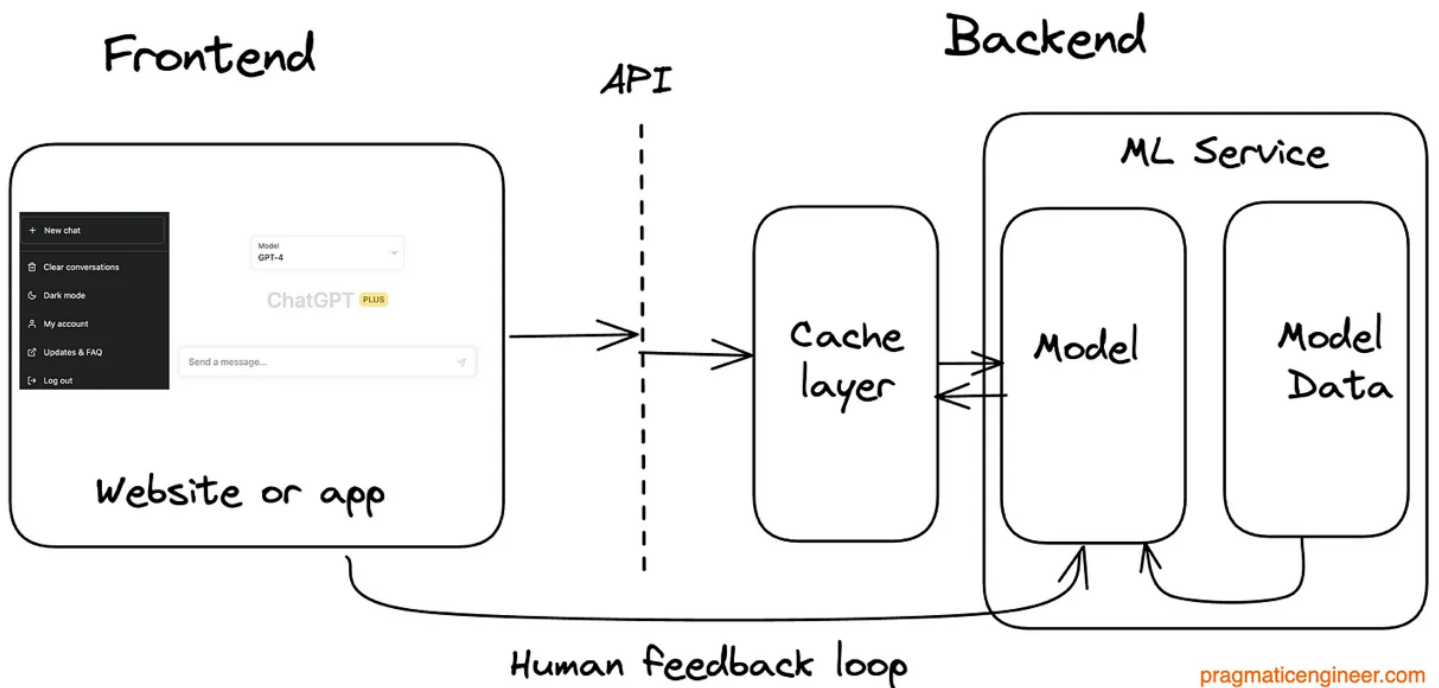
ChatGPT is presented through the chat.openai.com interactive interface:



The ChatGPT interactive interface

This interface is a web application which:

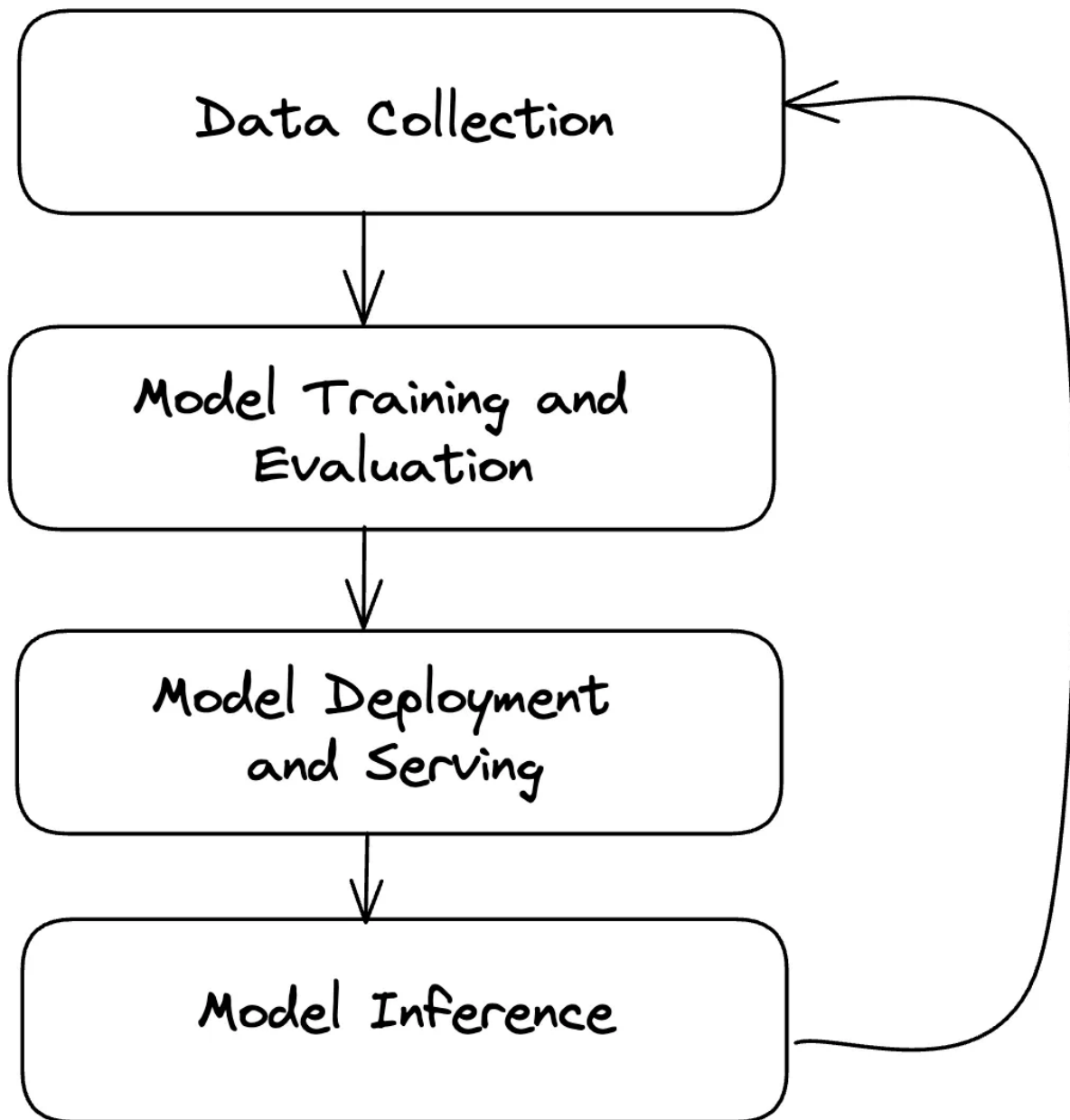
1. Takes user queries as input
2. Runs them through a ML model for inference, which means running live data points through an ML model to calculate the output.
3. Returns a set of output.
4. The app stores the state of that output and likely feeds it back into the model to generate further inference. This means the model continues a user's conversation session, if the user prompts it with further details. There are a couple of hypothesized ways they do this: Keep the length to 4000 tokens so that a number of queries could work together, or run the current discussion through summarization. The model itself doesn't remember the conversation but there are tricks they do to make it seem like it does.



The high-level structure of the ChatGPT app

Let's use the ML platform framework from the "What is Machine Learning" section to break down the ChatGPT app's functionality. As a reminder, the mental framework we use:

A Machine Learning Platform



pragmaticengineer.com

Typical components of a Machine Learning Platform

ChatGPT as an application has several components:

1. Data collection: First in ChatGPT, is data collection. ChatGPT was created by scraping an [enormous number of public data sources](#), including Wikipedia, [BookCorpus](#), and other sources, then synthesizing them into a dataset to be fed into an LLM. This dataset was frozen sometime in late 2021. All this data was cleaned and prepared for creating an LLM.

2. Model training and evaluation: This stage usually has software and human components and takes the longest in any model training process. For ChatGPT, the base model is a [Transformer model](#), which is a deep learning model using text-based inputs to generate outputs.

Transformers were initially used for machine translation, such as translating English to French, but have quickly become the bread and butter of many [natural language processing tasks](#) including text classification, question answering, and summarizing pieces of text, and much more.

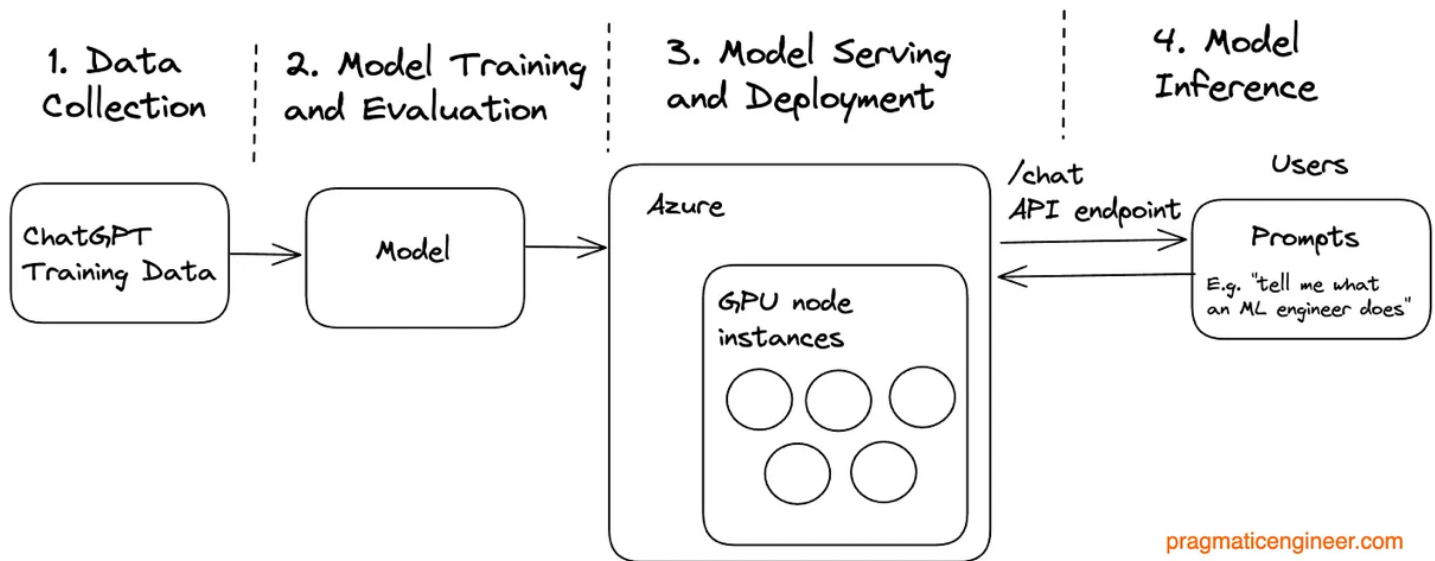
The [ChatGPT model includes two components](#): first, a Transformer model that's trained on the enormous set of input data and predicts the next word in a given sentence. And a second model based on reinforcement learning, in which a human takes a sampled prompt from a prompt dataset and picks the correct response for it.

These responses are fed back into the model and then prompts are fed back into the model, and the model generates several responses and a human selects the correct one. This data is fed into a second, reinforcement learning-based model, which then rewards "correct" results and downranks poor ones.

3. Model deployment and serving: Once the model is trained and fine-tuned, the model artifact, which includes all the model weights and the ability to perform model inference on new prompts, is deployed so users can enter a query into chat.gpt.com – or the recent product addition of the API – and get an answer to their query.

Instances of the model are [deployed on a distributed Kubernetes cluster](#) running in Azure, with open endpoints to the site based on authentication with an OpenAI API key.

4. Model inference: Finally the model API is skinned with a chat UI where you can talk to it directly. It stores previous chats, and users can give feedback on the model's accuracy, which is fed back into new model training approaches.



A high-level overview of ChatGPT as an application, using the Machine Learning Platform mental model

Machine learning is inherently non-deterministic. This is a major difference from software engineering, where if you create code to press a button, you can test if the button is pressed, every time. However, with ML you cannot guarantee the same result every time. Furthermore, in the case of deep learning models in particular, you can't even reverse-engineer all the features (inputs) that led the model to give output x or y.

The data-centric nature of ML means that, in addition to the usual things that can go wrong, such as service latency, there are a lot of components needed to keep an ML system running. In the paper, [Machine Learning: the high-interest credit card of technical debt](#), its authors at Google point out:

"Machine learning offers a fantastically powerful toolkit for building complex systems quickly. This paper argues that it is dangerous to think of these quick wins

as coming for free. Using the framework of technical debt, we note that it is remarkably easy to incur massive ongoing maintenance costs at the system level

when applying machine learning."

Anyone who has worked with ML systems knows this is the case; often you are not only troubleshooting non-working software components: model failures can be anything from unexpected [caching issues in production](#), a "classical" software engineering problem, or data drift which is when the properties of the model used for prediction change.

A common example of data drift is seasonality: you can't accurately predict the same volume of ice cream sales in summer as in winter. Often, these types of drifts are not very obvious and take a long time to detect. Null or malformed log data is also a cause for "silent" modeling failures.

Another property of the ML modeling process is that once you have more than one model, you're responsible for both their input data and metadata, which means setting up additional systems like feature stores and model stores to account for changes in the system.

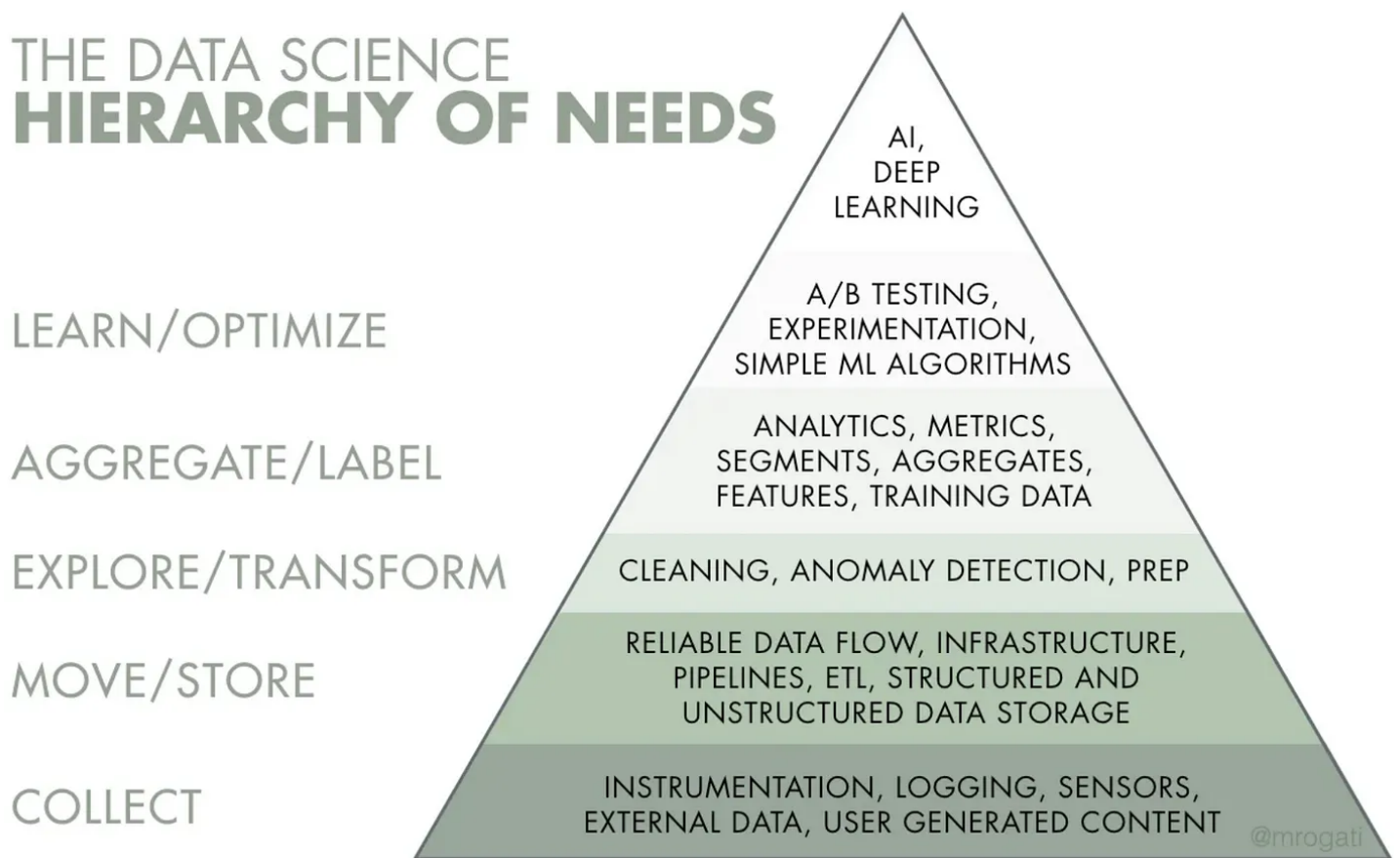
Here's a few examples of components in ML platforms which web apps usually don't include:

- **Feature stores:** functionality to pull data specifically formatted for building and serving models from the data warehouse, and do this quickly.
- **Monitoring data drift.** Data drift occurs when input data change over time. For example, when a customer base ages. It can also happen if a new technology wasn't included in the model's original assumptions; for example, cell phones' rapid spread across a telephony model built on landlines.
- **Model training monitoring:** the goal of this is to evaluate offline performance metrics to answer questions like how well a model will perform on unseen data.
- **Model registries.** These allow use of differently versioned model artifacts based on which is reformed for use in production.

which is preferred for use in production.

7. What does machine learning look like today?

We've covered deep learning, which makes major headlines but remains a very small part of today's data industry. However, we cannot get to ML without first putting data 'basics' in place. This diagram [developed by Monica Rogati](#) explains the baselines which need to be in place before we can have working ML models or AI applications:



The Data Science Hierarchy of Needs: a mental model of building to AI.

Source: [Monica Rogati](#)

Before companies can get to building ML models, they first need:

1. **Use cases.** Have specific use-cases for ML (not simply "do machine learning,") with real sets of rules that need to be inferred from data versus hand-coded as heuristics.

2. **Instrumentation.** Set up your application to collect the data you need for ML, such as logging the fields to be used as features. The heart of successful MLE projects is good data for modeling. This makes up the first bottom three parts of the triangle. If you don't have good, clean data that's collected, analyzed and prepared on a regular basis, then all your models will be junk and it won't matter whether they can be deployed or not.
3. **Collect data.** Have data flow into a centralized place via [Extract, Transform, Load](#) (ETL) for analysis, cleaning, and feature creation. This involves building good pipelines in conjunction with data engineering, infrastructure for maintaining pipelines, and good analysis environments in which analysts and ML engineers can explore the data before creating models for production.
4. **The algorithms.** With good data collected and analyzed, there is an ML use case, and we can build on this data to learn algorithms for how our product should work.
5. **Build models.** Use the data to build simple heuristics and models involving traditional tabular data. Once an ML engineer who specializes in modeling has a model, then the deployment story starts. This involves everything from setting up A/B testing, to track feature launches, to infrastructure for supporting model updates, data flows, and changes in the model's training data, or the input data used to create the model.
6. **Operationalize.** Once all this works end-to-end, operationalize it in ML platforms which offer the ability to do all these steps easily in a single place.

Today, the ML landscape is broken out into two fundamental pieces:

1. **Development:** work on developing ML models. It includes both the research and development of those models.
2. **Deployment:** work on deploying a wide variety of different models in production.

AI and deep learning make up the top of the pyramid, as complex work like this can only happen after all the other foundational work to build ML platforms is complete. It's important to bear in mind that deep learning makes sense for only a small subset of ML use-cases.

Most MLEs don't spend their time on deep learning. I see the importance of deep learning being exaggerated in the media, as I estimate only 5-7% of industry work is devoted to deep learning, in reality.

Further Reading

To keep pace with the rapidly changing, ever-evolving field of ML, here's a few resources I recommend:

- [Machine Learning: The High Interest Credit Card of Technical Debt](#): A classical canonical paper that goes into what makes up ML models, and what can go wrong.
- [Machine Learning Operations](#): A recent paper that discusses all the steps of an ML project.
- [Google's Rules for machine learning](#): a good overview of the field and things to think about as you become an MLE.

And also, I'd like to suggest thinking about [how to learn machine learning](#). I wrote a post on this topic recently, including three topics: learning the basics, learning how to model, and learning how to deploy models.

Takeaways

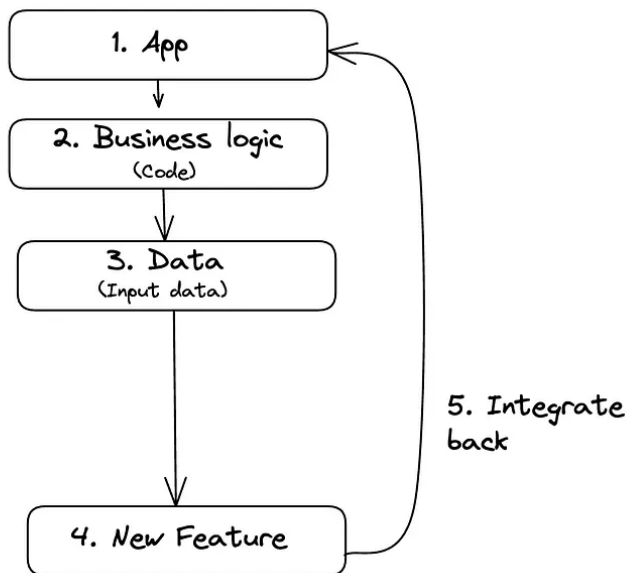
This is Gergely again.

Thanks very much, Vicki, for this overview of machine learning. If you'd like to connect with Vicki, you can [on Twitter](#) and [LinkedIn](#). Also, be sure to check out [her blog](#).

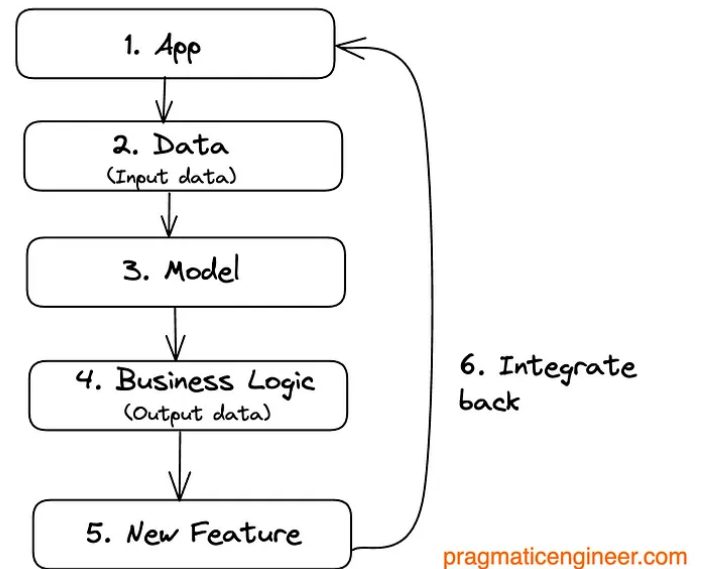
Software engineering and machine learning are related, but they're fundamentally different. ML is also built on top of coding; but beyond this, there's few parallels with traditional software engineering. Vicki really helps highlight the similarities and differences in visualizing a "typical" feature development process in both fields.

Putting them side by side:

Traditional software engineering feature development flow



ML engineering feature development flow



Building a new feature with "traditional" software engineering and with ML engineering

AI is all over the media these days, thanks to powerful advances led by OpenAI and ChatGPT, as well as with generative AI. There's lots of media speculation about what AI – or large language models – mean for the tech industry, and even humanity.

Machine Learning Engineering is a subset of AI applications, and the one from which innovations like LLMs emerge. However, as Vicki points out, ML engineers are rarely immersed in deep learning use cases, which relate to model training and evaluation. Instead, much of their work is in data collection, model serving and deployment and model inference.

I hope this article is useful. We're planning a follow-up later on, in which we'll go into more detail on the tools which machine learning engineers use, and how software engineers can transition into the ML field.